Nome: Marcelo Nassar de Souza da Silva, Anthoni

App Tinder — Trabalho 3

App de encontros em Jetpack Compose, com banco de dados local Room. Package: `com.example.myapplication`*.*

1. Visão geral

É um app estilo Tinder feito em Kotlin com Jetpack Compose. O usuário desliza perfis, curte pessoas, conversa, configura filtros e localização e pode comprar Boosts, Super Likes e planos pagando por Pix.

No Trabalho 3 o foco foi a **persistência**: as listas de **Curtidas** e **Conversas**, que antes viviam em memória (`mutableStateListOf`), passaram a ser gravadas no banco Room e continuam lá depois de fechar e reabrir o app.

2. Tecnologias

Item	Uso
Kotlin + Jetpack Compose (Material 3)	Interface
Navigation Compose	Navegação entre telas
Room 3 (`androidx.room3`) + KSP	Banco de dados local
`sqlite-bundled`	Driver SQLite embutido
Coroutines (`Dispatchers.IO`)	Operações de banco fora da thread principal
ZXing	Geração do QR Code do Pix
kotlinx.serialization	Serialização

Configuração: `minSdk 24`, `targetSdk 37`, Java 11.

3. Telas e navegação

A navegação é feita por um `NavHost` em `Rotas/AppNavegation.kt`. A barra de baixo (`BottomNavBar`) aparece nas telas principais e mostra um *badge* vermelho com o total de curtidas.

Tela	O que faz
Descobrir	Desliza perfis; curtir, favoritar ou passar. Curtir grava a

Tela	O que faz
	pessoa no banco. Tem abas "Pra você" e "Double Date"
Explorar / Explorar Detalhe	Perfis por categoria de relacionamento
Curtidas	Grade com as pessoas curtidas: adicionar, editar, favoritar, remover, ordenar
Detalhe da Curtida	Mostra a pessoa buscando-a no banco pelo id
Conversas	Lista de matches e de conversas: criar, renomear, remover
Detalhe da Conversa	Chat: busca conversa, pessoa e mensagens no banco; envia mensagens
Perfil	Resumo do usuário (total de curtidas e conversas) e atalhos
Configurações, Filtros	Preferências de busca
Localização	Modo Passaporte: locais salvos no banco
Astrologia	Compatibilidade por signo
Double Date / Config	Descoberta em dupla e suas opções
Assinatura, Compra, Pagamento	Planos, Boosts e Super Likes com cobrança por Pix e QR Code

4. Estrutura do código

```
app/src/main/java/com/example/myapplication/
├── MainActivity.kt        Carrega os dados do banco ao abrir o app
├── Rotas/                 Rotas.kt e AppNavegation.kt
├── modelo/Modelo.kt       Pessoa, Conversa, Mensagem (entities) e Perfil
├── dados/
│   ├── AppDados.kt        Catálogo fixo de perfis + totais em memória
│   ├── Preferencias.kt    Filtros e configurações
│   ├── Carteira.kt, Pix.kt, Astrologia.kt
│   └── Banco/             Tudo do Room (veja a seção 5)
├── telas/                 Uma tela por arquivo
└── componentes/           Avatar, BottomNavBar, BarraDeRolagem
```

5. Banco de dados (Room)

Tabelas

Tabela	Entity	Campos principais
`pessoas`	`Pessoa`	id, nome, idade, bio, fotoNome, distanciaKm, favorito

Tabela	Entity	Campos principais
`conversas`	`Conversa`	id, pessoaId, nome
`mensagens`	`Mensagem`	id, conversaId, texto, minha
`compras`	`CompraEntity`	id, tipo, item, quantidade, plano, valorCentavos, codigoPix, status, criadoEm, pagoEm
`carteira`	`CarteiraEntity`	id (sempre 1), superLikes, boosts, plano
`locais`	`LocalEntity`	id, nome, atual

As três primeiras são do Trabalho 3. As chaves primárias das tabelas novas são `Int` com `autoGenerate = true`.

DAOs

DAO	Operações
`PessoaDao`	`inserir`, `buscarTodas`, `buscarPorId`, `buscarPorNomeEIdade`, `contar`, `atualizar`, `deletar`
`ConversaDao`	`inserir`, `buscarTodas`, `buscarPorId`, `buscarPorPessoaId`, `contar`, `atualizar`, `deletar`
`MensagemDao`	`inserir`, `buscarPorConversa`, `ultimaDaConversa`, `excluirDaConversa`
`LojaDao`	Compras e carteira, com transações para confirmar pagamento e cancelar
`LocalDao`	Locais do Modo Passaporte

Todas as funções são `suspend`.

AppDatabase

`AppDatabase` está na **versão 2** e segue o padrão Singleton (`companion object` + `@Volatile` + `synchronized`), com driver `BundledSQLiteDriver` e contexto de consultas em `Dispatchers.IO`. O atalho `BancoDeDados.obter(contexto)` continua funcionando para o código antigo.

Repositórios

`SocialRepositorio` é o único ponto de acesso para curtidas, conversas e mensagens. Cada operação roda em `Dispatchers.IO` e tem `try/catch` com `Log.e`, então uma falha no banco não derruba a interface. Os repositórios de loja e de locais seguem o mesmo padrão.

6. Fluxo de dados

1. Ao abrir o app, a `MainActivity` chama `carregarInicial()`. Na primeira execução são gravados dois exemplos (Beatriz e Camila, com uma mensagem cada).
2. Cada tela busca no banco ao abrir (`LaunchedEffect`) e guarda o resultado em estado do Compose.

3. Depois de cada inserção, edição ou remoção, a tela relê o banco e a lista se atualiza.
4. O `SocialRepositorio` atualiza `AppDados.totalCurtidas` e `AppDados.totalConversas` para o badge da barra de baixo e o Perfil.

7. Decisões de projeto

- **Foto pelo nome do drawable.** A `Pessoa` guarda `"pessoa_1"` em vez de `R.drawable.pessoa_1`, porque o número do recurso pode mudar entre builds. A extensão `Pessoa.foto` converte o nome de volta no id.
- **Mensagens em tabela própria.** O Room não guarda lista numa coluna, então a lista de mensagens da `Conversa` virou a tabela `mensagens`.
- **Favorito como `Boolean`.** Antes era um `mutableStateOf` dentro da classe; agora favoritar é um `@Update`.
- **Sem duplicar curtidas.** Curtir o mesmo perfil duas vezes reaproveita a pessoa existente (`buscarPorNomeEIdade`).
- **Remoção em cascata manual.** Remover uma conversa apaga antes as mensagens dela.
- **Migração destrutiva.** Ao subir da versão 1 para a 2 o banco é recriado, o que é aceitável num app de estudo.

8. Como rodar

1. Abrir o projeto no Android Studio com SDK 37 instalado.
2. Aguardar o Gradle Sync (o KSP gera o código do Room).
3. Rodar em um emulador ou celular com Android 7.0 (API 24) ou superior.
4. Se uma versão antiga do app estiver instalada, desinstale antes.

9. Como testar a persistência

1. Em Curtidas, toque em **Adicionar curtida** e preencha os dados.
2. Edite a curtida (lápis) e marque-a como favorita (coração).
3. Em Conversas, crie uma conversa, envie mensagens e renomeie-a.
4. **Feche o app por completo** e abra de novo.
5. Tudo deve continuar como estava. Remova itens e repita para confirmar a remoção.

10. Limitações conhecidas

- Apagar uma curtida não apaga a conversa dela; a conversa fica com o nome e sem foto.
- O catálogo de perfis do Descobrir é fixo no código e não vem do banco.
- O pagamento por Pix é simulado: o código e o QR são gerados, mas não há integração com banco real.
- Todos os dados são locais; não há login, servidor ou sincronização.

Parte 2 — Detalhamento técnico

Esta parte aprofunda a Parte 1: o papel de cada arquivo, o dicionário de dados, as consultas SQL, os fluxos passo a passo, o tratamento de erros, o plano de testes e os problemas que apareceram durante o desenvolvimento.

11. Mapa de arquivos

Raiz do módulo `app`

Arquivo	Função
`MainActivity.kt`	Ponto de entrada. Num `LaunchedEffect(Unit)` carrega do banco a carteira, os locais salvos e os totais de curtidas/conversas, e depois exibe o `AppNavigation`
`build.gradle.kts`	Plugins (Compose, KSP, serialization), dependências do Room, navegação, ícones, ZXing e serialização

Pasta `modelo/`

Arquivo	Conteúdo
`Modelo.kt`	`Pessoa`, `Conversa` e `Mensagem` (entities do Room), a extensão `Pessoa.foto`, o objeto `Fotos` (nome do drawable <-> id) e a data class `Perfil` do catálogo do Descobrir

Pasta `dados/`

Arquivo	Conteúdo
`AppDados.kt`	Catálogo fixo de 5 perfis do Descobrir e os contadores `totalCurtidas` e `totalConversas`
`Preferencias.kt`	Filtros e configurações em memória (distância máxima, faixa etária, interesse, Double Date) e os locais do Modo Passaporte
`Carteira.kt`	Saldo em memória de Super Likes, Boosts e plano, espelhando a tabela `carteira`
`Pix.kt`	Geração do código Pix copia e cola usado no pagamento
`Astrologia.kt`	Dados de compatibilidade por signo

Pasta `dados/Banco/`

Arquivo	Conteúdo
`AppDataBase.kt`	`AppDatabase` (versão 2, Singleton) e o atalho

Arquivo	Conteúdo
	`BancoDeDados.obter()`
`Entidades.kt`	`CompraEntity`, `CarteiraEntity` e as constantes `TipoCompra` e `StatusCompra`
`LocalEntity.kt`, `LocalDao.kt`, `LocalRepositorio.kt`	Locais do Modo Passaporte
`LojaDao.kt`, `LojaRepoitorio.kt`	Compras e carteira
`PessoaDao.kt`, `ConversaDao.kt`, `MensagemDao.kt`	DAOs do Trabalho 3
`SocialRepositorio.kt`	Repositório de curtidas, conversas e mensagens, mais a classe `ResumoConversa`

Pasta `telas/` e `componentes/`

Cada tela fica em um arquivo (`TelaDescobrir.kt`, `TelaListaCurtidas.kt`, `TelaDetalheCurtidas.kt`, `TelaListaConversas.kt`, `TelaDetalheConversa.kt`, `TelaPerfil.kt` e as demais). Em `componentes/` ficam peças reutilizadas: `FotoOuInicial` (foto ou inicial do nome quando não há foto), `BottomNavBar` (barra de baixo com badge de curtidas) e `BarraRolagem` (barra de rolagem arrastável).

12. Dicionário de dados

Tabela `pessoas` (entity `Pessoa`)

Coluna	Tipo	Observação
`id`	Int	Chave primária, autogerada
`nome`	String	Obrigatório
`idade`	Int	Se o usuário não informar, a tela usa 18
`bio`	String	Mensagem ou descrição
`fotoNome`	String?	Nome do drawable, por exemplo `pessoa_1`. Nulo = sem foto (a tela mostra a inicial)
`distanciaKm`	Int?	Extraída do texto do perfil, por exemplo `a 2 km de distância` vira 2
`favorito`	Boolean	Marcado pelo coração

Tabela `conversas` (entity `Conversa`)

Coluna	Tipo	Observação
`id`	Int	Chave primária, autogerada
`pessoaId`	Int?	Pessoa da conversa. Nulo quando a conversa foi criada só com um nome
`nome`	String	Nome mostrado na lista e no topo do chat

Tabela `mensagens` (entity `Mensagem`)

Coluna	Tipo	Observação
`id`	Int	Chave primária, autogerada. Também define a ordem das mensagens
`conversaId`	Int	Conversa a que a mensagem pertence
`texto`	String	Conteúdo
`minha`	Boolean	`true` = enviada pelo usuário (balão à direita); `false` = recebida

Tabelas que já existiam

Tabela	Colunas	Observação
`compras`	id (Long), tipo, item, quantidade, plano, valorCentavos, codigoPix, status, criadoEm, pagoEm	`tipo` é BOOST, SUPERLIKE ou PLANO. `status` é PENDENTE, PAGO ou CANCELADO
`carteira`	id (sempre 1), superLikes, boosts, plano	Uma única linha com o saldo do usuário
`locais`	id (Long), nome, atual	Só um local pode ter `atual = true`

Relações: `conversas.pessoaId` aponta para `pessoas.id` e `mensagens.conversaId` aponta para `conversas.id`. Elas são mantidas pelo código, sem `ForeignKey` do Room.

13. Consultas SQL dos DAOs

PessoaDao

Função	Anotação / SQL
`inserir(pessoa)`	`@Insert`, devolve o id gerado
`buscarTodas()`	`SELECT * FROM pessoas ORDER BY id`
`buscarPorId(id)`	`SELECT * FROM pessoas WHERE id = :id`
`buscarPorNomeEIdade(nome, idade)`	`SELECT * FROM pessoas WHERE nome = :nome`

Função	Anotação / SQL
	`COLLATE NOCASE AND idade = :idade LIMIT 1`
`contar()`	`SELECT COUNT(*) FROM pessoas`
`atualizar(pessoa)`	`@Update`, usa a chave primária
`deletar(pessoa)`	`@Delete`

ConversaDao

Função	Anotação / SQL
`inserir(conversa)`	`@Insert`
`buscarTodas()`	`SELECT * FROM conversas ORDER BY id`
`buscarPorId(id)`	`SELECT * FROM conversas WHERE id = :id`
`buscarPorPessoaId(pessoaId)`	`SELECT * FROM conversas WHERE pessoaId = :pessoaId LIMIT 1`
`contar()`	`SELECT COUNT(*) FROM conversas`
`atualizar(conversa)`	`@Update`
`deletar(conversa)`	`@Delete`

MensagemDao

Função	Anotação / SQL
`inserir(mensagem)`	`@Insert`
`buscarPorConversa(conversaId)`	`SELECT * FROM mensagens WHERE conversaId = :conversaId ORDER BY id`
`ultimaDaConversa(conversaId)`	`SELECT * FROM mensagens WHERE conversaId = :conversaId ORDER BY id DESC LIMIT 1`
`excluirDaConversa(conversaId)`	`DELETE FROM mensagens WHERE conversaId = :conversaId`

14. Fluxos passo a passo

14.1 Abrir o app

1. A `MainActivity` monta a tela e dispara o `LaunchedEffect`.
2. `LojaRepositorio.carregarCarteira()` lê a carteira e copia para o objeto `Carteira`.
3. `LocalRepositorio.carregar()` cria o local São Paulo na primeira vez e sincroniza a lista de locais.

4. `SocialRepositorio.carregarInicial()` verifica a flag `seed_social_feito` nas SharedPreferences. Se ainda não existe e as tabelas estão vazias, grava Beatriz, Camila e uma mensagem em cada conversa. Depois marca a flag.
5. `atualizarContagens()` preenche `totalCurtidas` e `totalConversas`, e o badge da barra de baixo aparece.

A flag existe para que, se o usuário apagar todas as curtidas, os exemplos não voltem na próxima abertura.

14.2 Curtir um perfil no Descobrir

1. O usuário toca em curtir (ou favoritar) no card.
2. A tela dispara uma coroutine e já passa para o próximo perfil, sem esperar o banco.
3. `SocialRepositorio.curtir(perfil)` procura uma pessoa com o mesmo nome e idade.
4. Se não existe, insere uma `Pessoa` com a foto (pelo nome do drawable), a distância e a bio `Deu match com você agora!`.
5. Se existe e o usuário favoritou, faz um `@Update` marcando `favorito = true`. Caso contrário, reaproveita a existente.
6. Os totais são atualizados, e o badge muda.

No Double Date o mesmo processo vale para os dois perfis da dupla.

14.3 Adicionar, editar e remover uma curtida

Ação	O que acontece
Adicionar	O diálogo coleta nome, idade e mensagem. `adicionarPessoa` faz o `@Insert`, e a tela relê a lista
Editar	O lápis abre o mesmo diálogo preenchido. Ao salvar, `atualizarPessoa` faz o `@Update` com `copy(...)`
Favoritar	`alternarFavorito` inverte o campo e faz `@Update`
Remover	`removerPessoa` faz o `@Delete` e atualiza os totais

Depois de cada ação a tela chama `listarPessoas()` de novo, então o que se vê é sempre o que está no banco.

14.4 Abrir uma conversa

1. Ao tocar em um card de curtida (ou em um match na linha do topo de Conversas), chama-se `iniciarConversaComPessoa`.
2. O repositório procura uma conversa com aquele `pessoaId`. Se não acha, cria uma com o nome da pessoa.
3. A tela navega para `detalhe_conversa/{id}`.
4. A tela de detalhe recebe apenas o id e busca no banco a conversa, a pessoa dela e as mensagens.

14.5 Enviar uma mensagem

1. O usuário digita e toca em enviar (ou usa a tecla de envio do teclado).
2. O campo é limpo na hora.
3. `enviarMensagem` faz o `@Insert` com `minha = true`.
4. A tela relê as mensagens e rola até a última.

O texto de status Enviado aparece apenas abaixo da última mensagem enviada por você.

14.6 Renomear e remover uma conversa

Ação	O que acontece
Renomear	O lápis abre o diálogo com o nome atual. `renomearConversa` faz `@Update` com o nome novo, ignorando texto vazio
Remover	`removerConversa` apaga primeiro as mensagens (`excluirDaConversa`) e depois a conversa (`@Delete`), e atualiza os totais

14.7 Compra e pagamento por Pix

1. Em Assinatura ou Compra, o usuário escolhe o item. `criarCompra` gera um código Pix com um identificador único e grava a compra com status PENDENTE.
2. A tela de Pagamento exibe o código e o QR Code, gerado com ZXing.
3. Ao confirmar, `confirmarPagamento` roda numa transação: marca a compra como PAGO, credita o item na carteira e grava a carteira. Se a compra já não estiver pendente, nada é creditado de novo.
4. Cancelar só funciona enquanto a compra estiver pendente.
5. A transação garante tudo ou nada: se algo falha no meio, nada é gravado.

O pagamento é simulado, sem integração com banco.

15. Estado e coroutines

Padrão usado nas telas

```
val repositorio = remember { SocialRepositorio.obter(contexto) }
val escopo = rememberCoroutineScope()
var lista by remember { mutableStateOf<List<Pessoa>>(emptyList()) }

LaunchedEffect(Unit) { lista = repositorio.listarPessoas() }

fun alterar(operacao: suspend () -> Unit) {
    escopo.launch {
        operacao()
        lista = repositorio.listarPessoas()
    }
}
```

O banco é lido uma vez ao abrir a tela. Cada alteração roda numa coroutine e, ao terminar, relê a lista. Como a lista é um estado do Compose, a tela se redesenha sozinha.

Onde roda cada coisa

Tipo de trabalho	Thread
Consultas e escritas do banco	`Dispatchers.IO`, via `withContext` dentro do repositório
Atualizar estado da tela e os totais	Thread principal
Navegação depois de criar uma conversa	Thread principal, dentro da coroutine da tela

Os totais são calculados no IO e só depois atribuídos ao `AppDados`, para que o estado do Compose seja alterado na thread principal.

Estado de carregamento nos Detalhes

Nas telas de Detalhes existe um estado `carregando`. Enquanto a consulta não termina a tela fica vazia. Só se o item realmente não existir aparece a mensagem de que ele não existe mais. Sem isso, a mensagem piscaria a cada abertura.

16. Tratamento de erros

- Toda chamada ao banco passa pela função privada `io(padrao) { ... }` do `SocialRepositorio`.
- Ela muda para `Dispatchers.IO` e envolve a operação em `try/catch`.
- Em caso de exceção registra `Log.e("SocialRepositorio", ...)` e devolve um valor padrão: lista vazia, `null` ou `Unit`.
- Um `CancellationException` é repassado, e não engolido, para que sair de uma tela cancele a operação normalmente.
- Resultado: uma falha no banco aparece no Logcat, mas não derruba o app.

17. Plano de testes manuais

N	Caso	Passos	Resultado esperado
1	Dados de exemplo	Instalar e abrir o app limpo; ir em Curtidas e Conversas	Beatriz e Camila aparecem nas duas telas
2	Adicionar curtida	Adicionar curtida com nome, idade e mensagem	O card aparece e o badge aumenta
3	Persistência	Fechar o app por completo e abrir	A curtida continua lá
4	Editar curtida	Tocar no lápis, alterar o nome e salvar	O card mostra o nome novo, também depois de reabrir
5	Favoritar	Tocar no coração	O card passa a mostrar Favorito e mantém o estado ao reabrir

N	Caso	Passos	Resultado esperado
6	Remover curtida	Tocar na lixeira	O card some, o badge diminui e continua removido ao reabrir
7	Curtir no Descobrir	Curtir um perfil e ir em Curtidas	A pessoa aparece com foto
8	Não duplicar	Curtir o mesmo perfil de novo	Continua uma só pessoa
9	Nova conversa	Tocar no + e criar uma conversa	Ela aparece na lista e continua ao reabrir
10	Mensagens	Abrir uma conversa e enviar duas mensagens	Aparecem em ordem; ao reabrir o app continuam
11	Renomear	Tocar no lápis da conversa e salvar um nome novo	O nome muda na lista e no chat
12	Remover conversa	Tocar na lixeira da conversa	Ela some, junto com as mensagens
13	Abrir pelo match	Tocar numa pessoa da linha Novos matches	Abre a conversa existente, sem criar outra

18. Problemas encontrados e como foram resolvidos

Problema	Causa	Solução
Erros `Unresolved reference 'Banco'` e `LojaDao_Impl` no código gerado	O `LojaDao.kt` declarava o pacote `dados.Banco` (B maiúsculo) e os demais usavam `dados.banco`. O código gerado pelo Room se perdia	Padronizar tudo em `dados.banco` e remover os imports `dados.Banco.LojaDao`
Muitos erros vermelhos de uma vez	Um erro na base (o código gerado do banco) faz tudo que depende dele ficar vermelho	Olhar apenas o primeiro erro da aba Build e fazer Clean e Rebuild
App não abre e o Logcat vazio	O emulador (Android 17, API 37) ficou OFFLINE, então o app nem foi instalado	Cold Boot, ou criar um emulador novo com API 34 ou 35, ou usar um celular
Tela Something went wrong com `play.google.com`	A Play Store do emulador sem internet. Não tem relação com o app	Ignorar e abrir o app pelo ícone ou pelo botão Run
Possível crash ao abrir depois de subir a versão do banco	O banco antigo (versão 1) não bate com o schema novo	Desinstalar o app, ou usar a migração destrutiva já configurada
Foto errada ou ausente	O número `R.drawable.x` pode mudar	Guardar o nome do drawable

Problema	Causa	Solução
depois de recompilar	entre builds	(`fotoNome`) e converter com o objeto `Fotos`

19. Requisitos do enunciado e onde estão no projeto

Requisito	Onde
Dependências do Room e KSP	`app/build.gradle.kts`
Pelo menos 2 entities, com `id: Int` autogerado	`Pessoa`, `Conversa`, `Mensagem` em `Modelo.kt`
Pelo menos 2 DAOs com insert, query, update e delete, todos `suspend`	`PessoaDao` e `ConversaDao`
Consulta filtrada por campo além de `buscarTodas()`	`buscarPorId(id: Int)` nos dois DAOs
`AppDatabase` com Singleton	`AppDataBase.kt`
Operações em coroutine no `Dispatchers.IO`	`SocialRepositorio.io`
Tela de Detalhes lê do banco	`TelaDetalheCurtidas.kt` e `TelaDetalheConversa.kt`
Listas lidas do banco ao abrir	`TelaListaCurtidas.kt` e `TelaListaConversas.kt`
CRUD que persiste	Seção 14
`try/catch` com `Log.e`	Seção 16
README com como rodar e decisões	`README.md`

20. Melhorias futuras

- Usar `ForeignKey` com `onDelete = CASCADE` para que apagar uma conversa apague as mensagens sem código manual.
- Apagar a conversa junto quando a curtida for removida.
- Trocar a migração destrutiva por uma `Migration` que preserve compras e carteira.
- Expor as listas como `Flow`, para a tela atualizar sozinha sem precisar reler.
- Mover as chamadas para um `ViewModel`, e assim sobreviver a rotações de tela.
- Salvar o catálogo de perfis do Descobrir no banco.
- Testes automatizados dos DAOs com banco em memória.

21. Glossário

Termo	Significado

Termo	Significado
Room	Biblioteca do Android Jetpack que facilita usar o SQLite com classes Kotlin
Entity	Classe que representa uma tabela
DAO	Interface com as operações de leitura e escrita de uma tabela
KSP	Ferramenta que gera o código do Room durante o build
Singleton	Padrão em que existe uma só instância de um objeto no app
Coroutine	Forma de rodar trabalho em segundo plano sem travar a tela
`Dispatchers.IO`	Conjunto de threads próprio para entrada e saída, como banco e arquivos
`LaunchedEffect`	Função do Compose que roda uma coroutine quando a tela aparece
Pix copia e cola	Texto que representa um pagamento Pix, o mesmo conteúdo do QR Code
Migração	Regra para atualizar o banco de uma versão para outra

Parte 3 — Documentação do processo e das decisões

Esta parte responde às perguntas do enunciado sobre o processo do trio. Os textos são um rascunho baseado no que realmente aconteceu no projeto. **Revisem com as palavras do grupo** e **coloquem os prints e vídeos** nos pontos marcados como Evidência, porque o enunciado diz que texto sem o app funcionando não vale.

22. Como estava o projeto no Trabalho 2, e o que mudou para chegar até aqui?

Como estava. O app já tinha navegação real com `NavHost`, barra de baixo com badge, as telas Descobrir, Explorar, Curtidas, Conversas, Perfil, Configurações, Filtros, Localização, Astrologia e Double Date, e uma loja de Boosts, Super Likes e planos com pagamento por Pix. A loja e os locais do Modo Passaporte já estavam no Room. Já as listas de **Curtidas** e **Conversas** viviam no `object AppDados` em `mutableStateListOf`. Funcionavam enquanto o app estava aberto, mas ao fechar tudo se perdia.

O que mudou.

Antes	Depois
Curtidas em `AppDados.curtidas` (memória)	Tabela `pessoas` no Room
Conversas em `AppDados.conversas`, com a lista de mensagens dentro da classe	Tabelas `conversas` e `mensagens`
Detalhes recebiam o objeto pronto da lista	Detalhes recebem só o id e buscam no banco

Antes	Depois
Curtir no Descobrir adicionava na lista em memória	Curtir grava uma `Pessoa` no banco
Só adicionar e remover	CRUD completo: adicionar, editar, favoritar e remover
`AppDados` guardava as listas	`AppDados` guarda só o catálogo de perfis e dois contadores

Evidência a anexar: print da tela de Curtidas e de Conversas com os dados de exemplo, e print do código antigo (`mutableStateListOf` no `AppDados`) ao lado do novo.

23. Quais Entity, DAO e Database foram criados, e por que esses campos?

Entities

Entity	Campos	Por que esses campos
`Pessoa`	id, nome, idade, bio, fotoNome, distanciaKm, favorito	São exatamente os dados que os cards e o detalhe mostram. A foto é guardada pelo **nome** do drawable, porque o número `R.drawable.x` pode mudar quando o app é recompilado. O favorito virou `Boolean` simples, porque `mutableStateOf` dentro de uma entity não funciona no Room
`Conversa`	id, pessoaId, nome	Só o que identifica a conversa e a pessoa dela. As mensagens saíram da classe
`Mensagem`	id, conversaId, texto, minha	O Room não guarda lista numa coluna, então a lista virou uma tabela ligada à conversa. O campo `minha` diz de que lado fica o balão

Todas usam `@PrimaryKey(autoGenerate = true)` com `Int`.

DAOs. `PessoaDao` e `ConversaDao` têm os quatro métodos básicos (`@Insert`, `@Query`, `@Update`, `@Delete`), todos `suspend`. O `MensagemDao` tem inserir e consultas, porque uma mensagem não é editada nem apagada isoladamente neste app.

Database. `AppDatabase`, versão 2, com as seis entities (as três novas e as três de loja e locais que já existiam). Segue o padrão Singleton com `companion object`, `@Volatile` e `synchronized`.

Camada extra. Criamos o `SocialRepositorio`, que fica entre as telas e os DAOs. Ele concentra o `Dispatchers.IO` e o `try/catch` com `Log.e`, em vez de repetir isso em cada tela.

Evidência a anexar: print dos arquivos `Modelo.kt`, `PessoaDao.kt` e `AppDataBase.kt` abertos no Android Studio.

24. Como decidimos as queries do DAO?

Além dos quatro métodos básicos, pensamos em **o que cada tela precisa**:

Método	Motivo
`buscarTodas()`	As telas de Curtidas e de Conversas listam tudo ao abrir
`buscarPorId(id: Int)`	As telas de Detalhes recebem só o id pela navegação e precisam buscar o item
`buscarPorNomeEIdade(...)`	Evitar duplicar a mesma pessoa se o usuário curtir o mesmo perfil duas vezes
`buscarPorPessoaId(...)`	Reaproveitar a conversa que já existe com aquela pessoa, em vez de criar outra
`ultimaDaConversa(...)`	Mostrar o preview da última mensagem na lista de conversas
`excluirDaConversa(...)`	Ao remover uma conversa, apagar também as mensagens dela
`contar()`	Alimentar o badge da barra de baixo e o resumo do Perfil

Evidência a anexar: print do `PessoaDao.kt` mostrando as anotações `@Query`.

25. Qual foi a dificuldade em migrar de `mutableStateListOf` para o Room? O que quebrou e como resolvemos?

O que quebrou	Por quê	Como resolvemos
A tela não atualizava mais sozinha	Com `mutableStateListOf`, mudar a lista redesenhava a tela. Com o Room as funções são `suspend` e devolvem uma cópia	A tela lê o banco num `LaunchedEffect`, guarda o resultado num estado e relê a lista depois de cada alteração
Favoritar deixou de funcionar do jeito antigo	`favorito` era um `mutableStateOf` dentro da classe	Virou `Boolean` na entity, e favoritar agora é um `@Update` seguido de recarregar a lista
A lista de mensagens não cabia na entity	O Room não guarda listas em coluna	Criamos a tabela `mensagens` ligada à conversa por `conversaId`
A foto podia sumir ou trocar	Guardar `R.drawable.x` no banco é frágil entre builds	Passamos a guardar o nome do drawable e a converter na hora de exibir
A tela de Detalhes mostrava Não existe mais por um instante	A busca no banco leva um tempo e o item ainda era nulo	Criamos o estado `carregando` e só mostramos a mensagem se a busca terminou sem achar
Dezenas de erros vermelhos de uma vez (`Unresolved`	O `LojaDao.kt` usava o pacote `dados.Banco` (B maiúsculo) e o resto `dados.banco`. O código	Padronizamos tudo em `dados.banco` e fizemos Clean e Rebuild. O aprendizado foi olhar só o **primeiro** erro da aba Build

O que quebrou	Por quê	Como resolvemos
`reference 'Banco'`, `LojaDao_Impl`)	que o Room gera se perdia	
Risco de o app travar ao abrir depois de mudar o banco	Subimos o banco da versão 1 para a 2	Configuramos recriar o banco na troca de versão e desinstalamos o app antigo

Evidência a anexar: print do erro `Unresolved reference 'Banco'` e print do mesmo trecho já corrigido.

26. Teste de persistência: adicionar, fechar, reabrir

O enunciado pede **print ou vídeo**, e o vídeo é o mais forte. Gravem a tela do emulador ou do celular em um único vídeo curto:

1. Abrir o app e ir em **Curtidas**.
2. Tocar em **Adicionar curtida**, preencher nome, idade e mensagem e confirmar. Mostrar o card novo e o badge subindo.
3. **Fechar o app por completo**: abrir a tela de apps recentes e deslizar o app para fora.
4. Abrir o app de novo e ir em Curtidas: a curtida continua lá.
5. Repetir em **Conversas**: criar uma conversa, enviar uma mensagem, fechar e reabrir.

Outros vídeos curtos que valem a pena, um por etapa do processo:

Vídeo	O que mostrar
Editar	Tocar no lápis, mudar o nome, fechar e reabrir
Favoritar	Tocar no coração, fechar e reabrir
Remover	Tocar na lixeira, fechar e reabrir: continua removido
Curtir no Descobrir	Curtir um perfil e achá-lo em Curtidas depois de reabrir
Detalhes	Abrir uma conversa e mostrar que as mensagens vêm do banco

Evidência a anexar: o vídeo principal, ou ao menos 4 prints na sequência: antes, depois de adicionar, app fechado e depois de reabrir.

27. Algum integrante do trio teve dificuldade? Como resolveram?

Esta resposta é **do grupo**, então preencham com o que de fato aconteceu. Para ajudar, aqui vão problemas reais que apareceram no desenvolvimento e podem servir de base se tiverem sido vividos por vocês:

- **Erros de compilação em cascata.** Um único erro no pacote do banco fez o projeto inteiro ficar vermelho. A solução foi ler só o primeiro erro da aba Build e corrigir o nome do pacote.

- **Emulador offline.** O emulador com Android 17 (API 37) ficou OFFLINE, o app não instalava e o Logcat ficava vazio. A saída foi religar com Cold Boot, criar um emulador novo com API 34 ou 35, ou testar no celular real.
- **Tela Something went wrong no emulador.** Era a Play Store sem internet, sem relação com o app. Descobrir isso evitou perder tempo procurando um erro que não existia.

Modelo para preencher:

Integrante	Dificuldade	Como resolveram
(nome 1)	*(o que travou)*	*(o que fizeram: pesquisa, ajuda do colega, professor, documentação)*
(nome 2)		
(nome 3)		

Evidência a anexar: se possível, um print do erro e outro depois de resolvido.

28. Checklist de evidências antes de entregar

Item	Feito
Print das telas de Curtidas e Conversas com os dados de exemplo	[ ]
Print do código antigo (`mutableStateListOf`) e do novo	[ ]
Print de `Modelo.kt`, `PessoaDao.kt` e `AppDataBase.kt`	[ ]
Print do erro e da correção (pacote `Banco`/`banco`)	[ ]
Vídeo do teste de persistência (adicionar, fechar, reabrir)	[ ]
Vídeos curtos de editar, favoritar, remover e curtir no Descobrir	[ ]
Tabela do item 27 preenchida com os nomes do trio	[ ]
Prints e vídeos na pasta `docs/prints/` e referenciados no README (ou link dos Slides)	[ ]

Se preferirem montar a documentação em **Google Slides**, como o enunciado permite, usem esta Parte 3 como roteiro, um slide por pergunta, e coloquem o link no `README.md`.